
Software Requirements Specification

for

Green-Finance

Prepared by

Laukik Deshpande
Soham Gaikwad
Atharva Hule

2024800023
2024800030
2024800038

laukik.deshpande24@spit.ac.in
soham.gaikwad24@spit.ac.in
atharva.hule24@spit.ac.in

Instructor: Prof.Kirti Chaudhari

Course: Software Engineering

Date: 29/07/26

TABLE OF CONTENTS	2
1. INTRODUCTION	4
1.1 PURPOSE	4
1.2 SCOPE	4
1.3 DEFINITIONS, ACRONYMS AND ABBREVIATIONS	4
1.4 REFERENCES	5
1.5 OVERVIEW	6
 2. OVERALL DESCRIPTION	 7
2.1 PRODUCT PERSPECTIVE	7
2.2 PRODUCT FEATURES (SUMMARY)	7
2.3 USER PROFILE	8
2.4 LIMITATIONS	8
2.5 ASSUMPTIONS AND DEPENDENCIES	8
 3. SPECIFIC REQUIREMENTS	 9
3.1 FUNCTIONAL REQUIREMENTS	9
RF01 — USER / SERVICE AUTHENTICATION	9
RF02 — SIMULATED PAYMENT / TRANSACTION PROCESSING	9
RF03 — KERNEL-LEVEL ENERGY TELEMETRY	10
RF04 — METRICS SCRAPING AND STORAGE	10
RF05 — CARBON-PER-TRANSACTION CALCULATION	10
RF06 — REAL-TIME DASHBOARD VISUALIZATION	11
RF07 — ALERTING ON HIGH EMISSION / ENERGY SPIKES	11
RF08 — CONTAINERIZED DEPLOYMENT	12
RF09 — HISTORICAL DATA QUERY	12
RF10 — CONFIGURATION MANAGEMENT	12
3.2 NON-FUNCTIONAL REQUIREMENTS	13
3.3 EXTERNAL INTERFACE REQUIREMENTS	13
3.3.1 USER INTERFACES	13
3.3.2 HARDWARE INTERFACES	13
3.3.3 SOFTWARE INTERFACES	14
3.3.4 COMMUNICATION INTERFACES	14
3.4 PERFORMANCE REQUIREMENTS	14
3.5 DESIGN CONSTRAINTS	14
3.5.1 STANDARDS COMPLIANCE	14
3.5.2 HARDWARE LIMITATIONS	15
3.6 SOFTWARE SYSTEM ATTRIBUTES	15
3.6.1 SECURITY	15
3.6.2 MAINTAINABILITY	15
3.6.3 RELIABILITY	15
3.6.4 AVAILABILITY	15
3.6.5 PORTABILITY	15
3.6.6 USE-CASE DIAGRAM	16
3.7 OTHER REQUIREMENTS — METRICS & DATA	17
 4. APPENDICES	 18
APPENDIX A — HIGH-LEVEL ARCHITECTURE	18
APPENDIX B — TEAM ROLE DISTRIBUTION	18
APPENDIX C — OUT OF SCOPE FOR THIS MINI PROJECT	18

1 INTRODUCTION

1.1 Document Purpose

This document specifies the Software Requirements Specification (SRS) for Eco-Monitor: A Real-Time Energy Observability Framework for Sustainable Financial Microservices. It follows the structure recommended by the IEEE 830-1998 standard for Software Requirements Specifications. The purpose of this document is to clearly define the functional and non-functional requirements of the system so that developers, testers, and evaluators share a single, unambiguous reference for what is being built and why. It is intended for the project team (developers), the project guide/evaluators, and any future contributor who needs to understand the system's scope and design constraints.

1.2 Scope

Eco-Monitor is an observability platform that measures and visualizes the Real-time energy consumption and carbon intensity of a simulated set of financial (banking) microservices — specifically payment processing and authentication services. The system does not process real money or real bank data; it uses simulated FastAPI-based banking endpoints to generate realistic transaction load.

The product integrates three engineering aspects that are usually handled separately:

- Backend services — FastAPI microservices that simulate authentication and payment/transaction workloads.
- Energy telemetry — Kepler (Kubernetes-based Efficient Power Level Exporter), which uses eBPF to attribute kernel-level power draw to individual containers/processes.
- Observability — Prometheus for metrics scraping and storage, and Grafana for dashboards, culminating in a derived "Carbon-per-Transaction" metric.

The output is a working local (and optionally cloud) deployment, orchestrated with Docker/Kubernetes, that lets a user or evaluator observe, in near real time, how much energy and estimated CO₂ each transaction on the simulated banking platform costs, and which service is the primary contributor.

1.3 Definitions, Acronyms and Abbreviations

Term	Description
API	Application Programming Interface.
CI/CD	Continuous Integration / Continuous Deployment.
CPU	Central Processing Unit

eBPF	Extended Berkeley Packet Filter — a Linux kernel technology allowing safe, sandboxed programs to run in kernel space, used here for energy telemetry.
ESG	Environmental, Social, and Governance — a framework for measuring sustainability and ethical impact.
Kepler	Kubernetes-based Efficient Power Level Exporter — exports per-process/per-pod energy metrics.
PromQL	Prometheus Query Language, used to compute derived metrics such as Carbon-per Transaction.
RAPL	Running Average Power Limit — an Intel/AMD CPU interface that reports real hardware power consumption; required by Kepler for accurate readings on bare-metal Linux.
RF/FR	Functional Requirement.
RNF/NFR	Non-Functional Requirement.
JWT	JSON Web Token, used for authentication.
SRS	Software Requirements Specification.

1.4 References

-  IEEE, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, IEEE Standards Association, 1998.
-   FastAPI, *FastAPI Official Documentation*. Available: <https://fastapi.tiangolo.com> 
-  Kepler Project, *Kepler Documentation*. Available: <https://sustainable-computing.io> 
-  Prometheus Authors, *Prometheus Documentation*. Available: <https://prometheus.io/docs>  Grafana Labs, *Grafana Documentation*. Available: <https://grafana.com/docs>
-   Docker Inc., *Docker Documentation*. Available: <https://docs.docker.com>
-   ScienceDirect, "Financial Performance and Sustainability Metrics," Article No. S2444569X25002240. Available: <https://www.sciencedirect.com>
-   ResearchGate, "How Have Investors Changed the Face of a Firm's Financial Performance?", Publication No. 399563971. Available: <https://www.researchgate.net>

1.5 Overview

The remainder of this document is organized as follows: Section 2 gives an overall description of the product — its perspective, major features, intended users, limitations, and assumptions. Section 3 lists the specific functional and non-functional requirements, external interface requirements, performance requirements, design constraints, and system attributes in detail. Section 4 provides appendices covering the system architecture, database/metrics schema, and the team's role distribution.

2. OVERALL DESCRIPTION

2.1 Product Perspective

Eco-Monitor is a new, self-contained system built for an academic Mini Project (TE Sem V, SPIT). It is not a component of a larger existing system, but it is designed so that its monitoring layer (Kepler + Prometheus + Grafana) could, in principle, be attached to any containerized workload beyond the simulated banking services used here.

At a high level, the system consists of four cooperating layers:

- Simulated Banking Layer — FastAPI services (auth-service, payment-service) that generate realistic request load.
- Energy Telemetry Layer — Kepler running as a privileged component that reads kernel/RAPL energy counters via eBPF and attributes them to each service's containers.
- Metrics Aggregation Layer — Prometheus scrapes both the application metrics (request counts, latency) and Kepler's energy metrics on a fixed interval.
- Visualization Layer — Grafana dashboards join these metric streams via PromQL to compute and display the Carbon-per-Transaction metric and per-service energy hotspots.

2.2 Product Features (Summary)

- Simulated authentication and payment/transaction endpoints (FastAPI) that produce a realistic, controllable request load.
- Real-time, kernel-level energy attribution per microservice using Kepler's eBPF-based instrumentation.
- Centralized metrics collection and storage via Prometheus, including custom PromQL queries.
- A Grafana dashboard displaying live "Carbon-per-Transaction" and per-service energy consumption over time.
- Containerized deployment via Docker Compose (and optionally Kubernetes) so the whole stack runs with a single command.
- Identification of energy-inefficient "hotspot" services/endpoints to support ESG-oriented engineering decisions.

2.3 User Profile

Type of User	Technical Background	Primary Activities
Developer / Project Team	Backend (FastAPI, Python), DevOps (Docker, Kubernetes), Monitoring (Prometheus, Grafana, Kepler)	Build, deploy, and maintain the simulated services and the monitoring stack.
Evaluator / Guide	Software Engineering, Cloud Computing background	Review dashboards, verify Carbon-per Transaction accuracy, assess system design.
End Viewer (Dashboard User)	Basic — no coding required	View Grafana dashboards to observe energy and carbon trends.

2.4 Limitations

- Kepler's most accurate energy readings depend on Intel/AMD RAPL hardware counters available only on bare-metal Linux; under virtualized environments such as WSL2, the system falls back to Kepler's machine-learning-based power estimator, which provides lower accuracy.
- The banking services (authentication, payments) are simulated for demonstration purposes and are not connected to any real financial institution or real money movement.
- The system is designed to run on a single developer machine or a small local cluster; it is not load tested for production-scale financial transaction volumes.
- Energy-to-carbon conversion uses a configurable regional emission factor (grams CO₂ per kWh), which is an approximation rather than a live grid-carbon-intensity feed.
- The project timeline (one academic semester) constrains the scope to Phase 1 of the broader Green-Finance vision; Kafka-based event streaming and ML-driven portfolio rebalancing are out of scope for this Mini Project.

2.5 Assumptions and Dependencies

- The development and demonstration machine has Docker (and, where applicable, a Linux kernel with eBPF support) available.
- Requirements described in this document are assumed to be stable for the duration of the Mini Project I evaluation cycle.
- Prometheus and Grafana are available as open-source components with no licensing cost.
- The simulated transaction load is assumed to be representative enough of real banking request patterns for the purpose of demonstrating the Carbon-per-Transaction metric.

3. SPECIFIC REQUIREMENTS

3.1 Functional Requirements

Each functional requirement below follows the IEEE 830 requirement-card format: ID, name, characteristics, description, related non-functional requirements, and priority.

RF01 — User / Service Authentication

Requirement ID	RF01
Requirement Name	Authentication Service
Characteristics	Simulated banking users must authenticate before invoking payment endpoints.
Description	The system shall allow the authentic service (FastAPI) to expose /register and /login endpoints. On successful login it issues a JWT access token that must be attached to subsequent payment-service requests.
Related Non-Functional Requirements	RNF01, RNF02, RNF05, RNF08
Priority	High

RF02 — Simulated Payment / Transaction Processing

Requirement ID	RF02
Requirement Name	Payment Service
Characteristics	The system must generate realistic transaction load to observe under the energy monitor.
Description	The system shall allow the payment service to expose an endpoint that accepts a simulated transaction (amount, sender, receiver) and records it, generating measurable CPU/network activity that Kepler can attribute an energy cost to.
Related Non-Functional Requirements	RNF01, RNF05, RNF07
Priority	High

RF03 — Kernel-Level Energy Telemetry

Requirement ID	RF03
Requirement Name	Kepler Energy Exporter
Characteristics	The system must capture real (or estimated) energy usage per running container/service.
Description	The system shall allow Kepler to deploy alongside the services and uses eBPF probes to read RAPL energy counters (bare-metal Linux) or its built-in ML-based estimator (virtualized environments such as WSL2) and exports these as Prometheus-compatible metrics.
Related Non-Functional Requirements	RNF03, RNF05, RNF07
Priority	High

RF04 — Metrics Scraping and Storage

Requirement ID	RF04
Requirement Name	Prometheus Aggregation
Characteristics	The system must collect and retain time-series metrics from all services.
Description	The system shall allow Prometheus to configure with scrape jobs for authentication service, payment service, and Kepler, polling each at a fixed interval and storing the resulting time series for querying.
Related Non-Functional Requirements	RNF05, RNF06
Priority	High

RF05 — Carbon-per-Transaction Calculation

Requirement ID	RF05
Requirement Name	Carbon Metric Computation
Characteristics	The system must derive a human-meaningful sustainability metric, not just raw watts.

Description	The system shall allow a PromQL query to combine Kepler's energy-per-service metric with the payment-service's transaction-count metric, multiplies by a configurable regional carbon-intensity factor, and produces a "grams CO ₂ per transaction" value.
Related Non-Functional Requirements	RNF01, RNF05
Priority	High

RF06 — Real-Time Dashboard Visualization

Requirement ID	RF06
Requirement Name	Grafana Dashboard
Characteristics	Results must be viewable without querying Prometheus directly.
Description	The system shall allow a Grafana dashboard to display live graphs for: total energy per service, Carbon-per-Transaction over time, and a ranked list of the most energy-intensive services ("hotspots").
Related Non-Functional Requirements	RNF01, RNF02, RNF05
Priority	High

RF07 — Alerting on High Emission / Energy Spikes

Requirement ID	RF07
Requirement Name	Threshold Alerts
Characteristics	Users should be notified when a service's carbon cost exceeds a defined threshold.
Description	The system shall allow Grafana (or Prometheus Alertmanager) to raise a visual/log alert when Carbon-per-Transaction exceeds a configurable threshold for a sustained period.
Related Non-Functional Requirements	RNF05, RNF07
Priority	Medium

RF08 — Containerized Deployment

Requirement ID	RF08
Requirement Name	Docker Compose Orchestration
Characteristics	The full stack must be reproducible with minimal manual setup.
Description	The system shall a single docker to compose up command starts auth-service, payment-service, Kepler, Prometheus, and Grafana with pre-wired networking and configuration.
Related Non-Functional Requirements	RNF03, RNF06, RNF08
Priority	High

RF09 — Historical Data Query

Requirement ID	RF09
Requirement Name	Time-Range Queries
Characteristics	Users must be able to inspect past energy/carbon trends, not only the current value.
Description	The system shall allow Grafana dashboards to allow the user to select a custom time range (e.g., last hour, last day) over which Prometheus-stored metrics are visualized.
Related Non-Functional Requirements	RNF01, RNF05
Priority	Medium

RF10 — Configuration Management

Requirement ID	RF10
Requirement Name	Environment Configuration
Characteristics	Deployment parameters (ports, carbon factor, scrape intervals) must be externally configurable.
Description	The system shall allow all environment specific values to be read from a .env file / Prometheus config file rather than hard-coded, allowing the same codebase to run in different environments.
Related Non-Functional Requirements	RNF03, RNF06, RNF08

Priority	Medium
----------	--------

3.2 Non-Functional Requirements

ID	Name	Description	Priority
RNF01	Usability	Dashboards and API responses must be simple and understandable without extensive training.	High
RNF02	Help / Documentation	The project must include a README with setup instructions and endpoint documentation.	High
RNF03	Maintainability	Code must be modular (separate services, clear folder structure) so components can be updated independently.	High
RNF04	Portability	The system must run via Docker on any machine with Docker installed, independent of host OS specifics.	Medium
RNF05	Performance	API response times should remain under 2 seconds under simulated load; metrics scraping must not materially affect service performance.	High
RNF06	Reliability	The monitoring stack must not lose metrics data during normal operation; services should recover cleanly after a restart.	High
RNF07	Availability	During a demonstration/evaluation session, all services should remain continuously available.	Medium
RNF08	Security	Authentication must use JWT tokens; secrets (API keys, DB credentials) must not be hard-coded and must be excluded from version control via .gitignore.	High

3.3 External Interface Requirements

3.3.1 User Interfaces

The primary user interface is the Grafana web dashboard, accessed through a standard web browser. It presents graphs, single-stat panels, and tables for energy and carbon metrics. A secondary interface is the FastAPI-generated Swagger UI, used by developers to test the auth-service and payment-service endpoints directly.

3.3.2 Hardware Interfaces

- A development/demonstration machine with a minimum of 8 GB RAM.

- For fully accurate Kepler readings: bare-metal Linux with Intel/AMD RAPL support. On Windows via WSL2, Kepler's ML-based estimator is used instead of direct RAPL reads.
- Standard network adapter for local API/dashboard access.

3.3.3 Software Interfaces

Component	Technology
Backend services	Python, FastAPI
Energy telemetry	Kepler (eBPF-based exporter)
Metrics storage	Prometheus
Visualization	Grafana
Containerization	Docker, Docker Compose (Kubernetes optional)
Operating System	Linux (bare-metal, recommended) or Windows with WSL2

3.3.4 Communication Interfaces

- REST APIs over HTTP(S) between the frontend/Swagger UI and the FastAPI services.
- Prometheus scrapes metrics endpoints (/metrics) over HTTP at a fixed polling interval.
- Grafana communicates with Prometheus via its HTTP query API (PromQL).

3.4 Performance Requirements

- API endpoints (auth-service, payment-service) shall respond within 2 seconds under simulated load.
- Prometheus scrape interval should not exceed 15 seconds so that dashboards feel "Real-Time."
- The monitoring layer's own resource overhead (Kepler, Prometheus, Grafana) should remain a small fraction of total system resource usage so it does not itself distort the measurements it is trying to take.

3.5 Design Constraints

3.5.1 Standards Compliance

- This SRS follows IEEE 830-1998 for Software Requirements Specifications.
- REST API design follows standard HTTP method/status-code conventions.

3.5.2 Hardware Limitations

- Full RAPL-based Kepler accuracy is only available on bare-metal Linux hardware exposing Intel/AMD energy counters; this is a known constraint documented for evaluators.

3.6 Software System Attributes

3.6.1 Security

Authentication uses JWT access tokens issued by the auth-service. Secrets (database credentials, API keys) are managed via environment variables and excluded from version control using .gitignore.

3.6.2 Maintainability

The codebase is organized into clearly separated modules (auth, payment/ledger, carbon-calculator, monitoring configuration) so that each concern can be modified independently, in line with the team's Eco-Monitor project structure conventions.

3.6.3 Reliability

Docker Compose ensures that services restart automatically on failure during a demonstration, and Prometheus's local time-series storage persists metrics across short restarts.

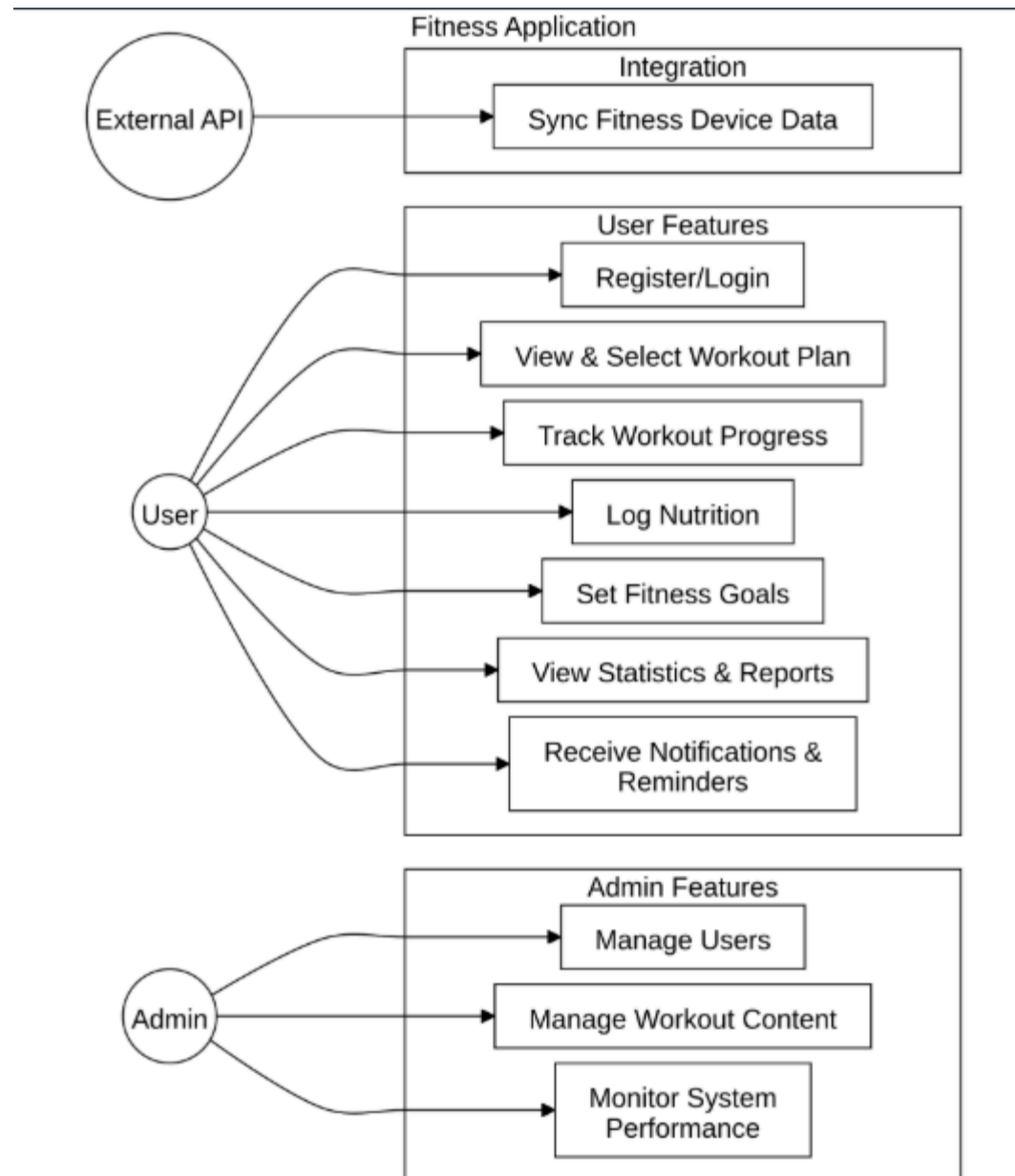
3.6.4 Availability

The system targets continuous availability throughout any demonstration or evaluation session; no scheduled downtime is required for a project of this scope.

3.6.5 Portability

The entire stack is containerized with Docker, allowing it to run on any machine with Docker installed, independent of the underlying host operating system (with the RAPL/ML-estimator caveat noted above).

3.6.6 Use-Case Diagram



3.7 Other Requirements — Metrics & Data

Metric / Data Item	Source	Description
kepler_container_joules_total	Kepler	Cumulative energy (in joules) attributed to a container/service.
http_requests_total	FastAPI services (Prometheus client)	Count of requests handled per endpoint, used as the transaction denominator.
carbon_per_transaction	Derived (PromQL)	Energy per service × regional carbon-intensity factor ÷ transaction count.
http_request_duration_seconds	FastAPI services	Latency distribution used to verify the 2- second performance requirement.

4. APPENDICES

Appendix A — High-Level Architecture

Frontend/Swagger UI → API layer (auth-service, payment-service) → Kepler (eBPF energy telemetry, sidecar to each service) → Prometheus (metrics aggregation) → Grafana (Carbon-per-Transaction dashboard).

Data flow for a simulated transaction: a client calls the payment-service endpoint → the service processes the request and increments its request counter → Kepler continuously attributes CPU/energy draw to the payment-service container → Prometheus scrapes both the request counter and Kepler's energy metric →

a PromQL rule combines them into Carbon-per-Transaction → Grafana renders the result on a live dashboard.

Appendix B — Team Role Distribution

Member	UID	Primary Responsibility
Atharva Pramod Hule	2024800038	Frontend dashboard development, system documentation, and integration support to ensure seamless interaction between application components.
Laukik Vidyadhar Deshpande	2024800023	Testing, carbon-factor research, and report generation.
Soham Haridas Gaikwad	2024800030	Backend (FastAPI services), Monitoring stack (Prometheus/Grafana/Kepler configuration), DevOps (Docker Compose orchestration).

Appendix C — Out of Scope for This Mini Project

- Kafka-based asynchronous event streaming between services.
- ML-driven portfolio rebalancing and Real-Time market/trading features.
- Kubernetes-based autoscaling and multi-node cloud deployment.
- Integration with any real banking or payment provider.

Appendix D — Document Control

Date	Revision	Author	Description
28-07-2026	1.0	Atharva Pramod Hule	Initial IEEE 830 SRS for Eco Monitor Mini Project I.

